

Distributed Algorithms in ISC

A core aspect of how ISC works is through the use of distributed algorithms (i.e. algorithms that run on multiple nodes in a network and require communication between them). These algorithms (also sometimes called *protocols*) are used for various purposes, such as reaching consensus on the state of the system, managing access to resources, and synchronizing data between nodes.

A recommended reading to understand the design and implementation of distributed algorithms in general is the book *Fault-tolerant message-passing distributed systems* by Michel Raynal [1]. It is not necessary to read the whole book, but it is recommended to read at least the first few chapters to understand the basics of distributed algorithms and the different types of well-known algorithms that exist.

Contents

1. GPA	2
1.1. The GPA interface	3
1.2. OwnHandler and AckHandler	4
1.2.1. OwnHandler	4
1.2.2. AckHandler	4
1.3. Alternative implementation	4
2. TLA+ specifications	5
3. Chains and chainNodeImpl	5
4. AccessMgr	5
4.1. Architecture	5
4.2. AccessMgr GPA	6
4.2.1. State	6
4.2.2. Inputs	7
4.2.3. Message exchange	7
4.2.4. Convergence	7
5. ChainMgr	8
5.1. State	8
5.2. Inputs	8
5.3. Messages	9
5.4. HandleCommitteeLogOutput	9
5.5. CommitteeLog	9
6. StateMgr	10
6.1. Outer wrapper	10
6.2. StateMgr GPA	10
6.2.1. Inputs	10
6.2.2. Messages	11
7. Mempool	11
7.1. Outer wrapper	11
7.2. Mempool GPA	11
7.2.1. Inputs	11
7.2.2. Messages	12
8. Consensus	12
8.1. DSS	14
8.2. NonceDKG	15
8.3. ACSS	15

8.4. RBC	16
8.5. ACS	16
8.6. ABA	16
8.7. CC	17
References	17

1. GPA

The distributed algorithms in ISC are implemented using a framework called GPA, which stands for “General Purpose Algorithm”. GPA provides a common interface and structure for implementing distributed algorithms in ISC. It has the following design principles:

- **Pseudo-functional:** Message-passing is implemented as methods that receive some input (from the same node) or message (from some other node), and return some messages (if any) to be sent to other nodes. It is not really functional, because the state of the algorithm is stored in the struct that implements the GPA interface, and is mutated by the methods. However, it is functional in spirit because the methods do not have side effects on other parts of the system, and only mutate the state of the algorithm itself.
- **Single-threaded:** Each GPA instance and all its subcomponents (if any) run in the same goroutine as the caller, and all input and message calls are blocking. This eliminates the need to worry about concurrency issues. In practice this makes the code more complex, because Go is designed for concurrent programming, and it is often more natural to write code that uses goroutines and channels.
- **Common testing framework:** All GPA implementations can be tested using the same testing tool, called [TestContext](#).

At any point in time there may be several GPA instances running on the same node, and they can be organized in a hierarchical structure. All nodes have the same hierarchical structure of GPA instances, and each GPA instance communicates with the corresponding GPA instance on other nodes. Figure 1 shows an example of this behavior.

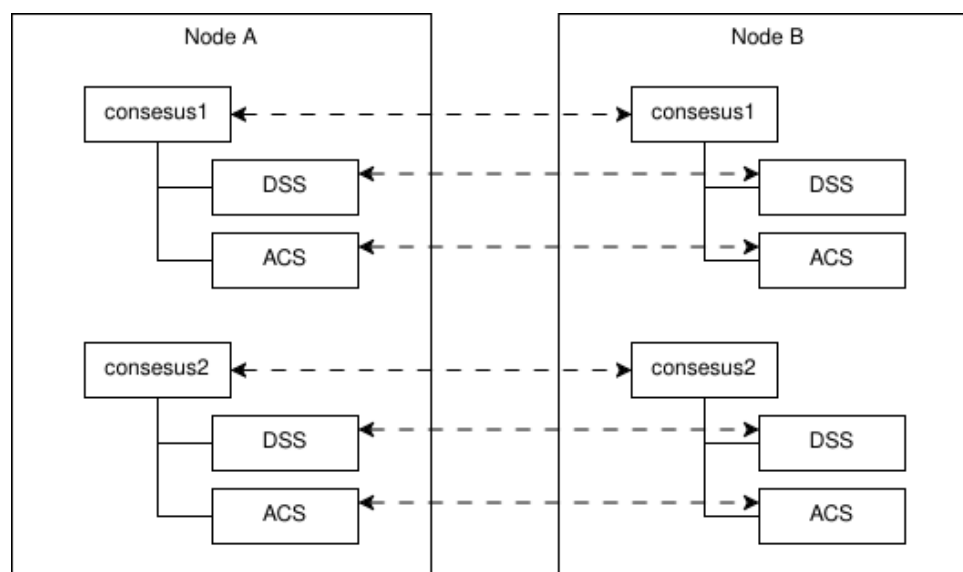


Figure 1: Example: many GPA instances running in two nodes.

- `Chains` (container for all chains managed by the node)
 - `AccessMgr` (which nodes has access to which chain? who can access you? inform other nodes which chains you run, share information to access nodes)
 - `GPA:AccessMgr`
- `chainNodeImpl` (one for each chain managed by the node – implements `chain.Chain` – main container for all objects & logic needed to run a chain)
 - `GPA:ChainMgr`
 - `GPA:CommitteeLog` (one for each committee the node participates in – node might be lagging or ahead; for a node to be in sync decide which Consensus instances to work on, and propose the base anchor)
 - `StateMgr`
 - `GPA:StateMgr` (make sure that the node has all the blocks it needs e.g. after reboot)
- `Mempool`
 - `GPA:Mempool` (share requests between nodes)
- `consensusrunner` (one for each currently running Consensus instance)
 - `GPA:Consensus`
 - `GPA:DSS` (DSS / Distributed Schnorr Signature - signs the produced block)
 - `GPA:NonceDKG` (Nonce distributed key generation)
 - `GPA:ACSS` (Asynchronous Complete Secret Sharing – one for each node in the committee)
 - `GPA:RBC` (Reliable Broadcast – Bracha’s algorithm)
 - `GPA:ACS` (Asynchronous Common Subset – from the Honey Badger paper)
 - `GPA:RBC` (Reliable Broadcast – Bracha’s algorithm – one for each node in the committee)
 - `GPA:ABA` (Asynchronous Binary Agreement – Mostefaoui’s algorithm – one for each node in the committee)
 - `GPA:CC` (Common Coin based on a BLS Threshold signatures – one for each round)

Listing 1: Hierarchy of GPA implementations in ISC. Some of the listed components are not GPA implementations, but rather “containers” or “managers”. Actual GPA implementations are marked with `GPA:`.

Listing 1 shows all implementations of the GPA interface in ISC.

1.1. The GPA interface

All GPA instances are objects that implement the GPA interface, which models any distributed algorithm that:

- may accept inputs, and in consequence send some messages to peers
- may receive messages, and in consequence send some messages to peers
- will eventually produce some output

The difference between an “input” and a “message” is that the former is generated by the same node (e.g. a timer, or a request from another component), while the latter is generated by other nodes in the committee.

```
type GPA interface {
    // must be called to pass some input to the algorithm
```

```

Input(inp Input) OutMessages
// must be called whenever a message is received from a peer
Message(msg Message) OutMessages
// returns the output of the algorithm, if it has produced one
Output() Output
}

```

1.2. OwnHandler and AckHandler

GPA instances produce outgoing messages addressed to other nodes, but the GPA interface itself says nothing about *how* those messages are delivered. Two reusable GPA wrappers handle common delivery concerns: `OwnHandler` and `AckHandler`. They are typically composed together (with `OwnHandler` as the inner layer and `AckHandler` as the outer layer) so that the wrapped GPA benefits from both.

1.2.1. OwnHandler

`OwnHandler` intercepts outgoing messages whose recipient is the local node and feeds them back into the wrapped GPA as ordinary incoming messages, looping until no more self-addressed messages are produced. This means the actual GPA implementation never needs to special-case “messages to self” – it can simply send a message to any node (including itself), and `OwnHandler` takes care of short-circuiting the delivery. Messages addressed to other nodes pass through unchanged.

1.2.2. AckHandler

`AckHandler` wraps a GPA to provide a **reliable channel** abstraction over an unreliable network. It ensures every message is eventually delivered by resending unacknowledged messages periodically, and piggy-backing acknowledgements on outgoing traffic.

The key mechanisms are:

- **Batching:** outgoing messages to the same peer are grouped into a single `ackHandlerBatch`, which is assigned a monotonically increasing batch ID.
- **Acknowledgements:** each batch carries an `acks` field listing the IDs of previously received batches from that peer. This way, acknowledgements travel for free inside regular traffic. If there is no outgoing traffic, the tick handler will trigger resends which implicitly carry the pending acks.
- **Resending:** a periodic tick input (produced by `MakeTickInput`) checks all unacknowledged batches. Any batch older than the configured `resendPeriod` is resent. Once an ack for a batch is received, it is removed from the unacknowledged set.
- **Peer initialization:** before sending payload batches to a peer for the first time, `AckHandler` performs a lightweight handshake (`ackHandlerReset`) to learn the peer’s latest known batch ID. This avoids ID collisions after restarts. Messages are queued in `initPending` until the handshake completes.
- **Deduplication:** received batch IDs are tracked in `recvAcksIn`, so duplicate deliveries (due to resends) are detected and not forwarded to the nested GPA a second time.

1.3. Alternative implementation

There is an experimental alternative implementation of some of the distributed algorithms called `actors`, in which the single-threaded and pseudo-functional design principles of GPA are lifted, and the algorithms are implemented using Go’s concurrency primitives such as goroutines and channels. As a result, the code tends to be more straightforward and easier to read. The Consensus protocol and all its subcomponents are implemented using actors, although it has not been thoroughly tested and so it is currently unsuitable for production use.

2. TLA+ specifications

[TLA+](#) is a formal specification language designed by Leslie Lamport for modeling and verifying concurrent and distributed systems. A TLA+ specification describes a system as a state machine: an initial state predicate together with a next-state relation over a set of variables. Properties such as safety invariants and liveness conditions are expressed in temporal logic, and can be verified automatically using the TLC model checker (for finite instances) or mechanically proved using the TLAPS proof system. Writing a TLA+ specification forces the author to think precisely about the algorithm's correctness before (or alongside) implementing it in code, and the model checker can find subtle bugs that are difficult to catch with testing alone.

Several of the distributed algorithms in ISC have accompanying TLA+ specifications. These live alongside the Go implementation in the same package directory. Listing 2 lists all available TLA+ specifications.

3. Chains and chainNodeImpl

The Chains component is the main container for all chains managed by the node. It is responsible for creating and managing the lifecycle of chains, and for providing access to them. It contains:

- AccessMgr: responsible for managing access to chains
- one chainNodeImpl for each chain managed by the node, which contains:
 - ChainMgr, responsible for managing the lifecycle of the chain
 - StateMgr, responsible for managing the chain state (aka the DB)
 - Mempool, responsible for managing the chain requests
 - one Consensus instance for each currently running Consensus instance

4. AccessMgr

This component is responsible for keeping track of trusted nodes and which nodes have access to which chains. There is exactly one AccessMgr per Wasp node (as opposed to one per chain), and it is owned by Chains.

Each node can designate other nodes as **access nodes** for the chains it manages. However, the access nodes have no way of knowing from their own configuration alone which nodes consider them access nodes. The purpose of AccessMgr is to solve this problem: it runs a distributed protocol that informs each access node which nodes will act as **servers** for it.

The design goals are:

- Nodes not related to a chain should get no information about it.
- The list of server nodes should be *transient* (not persisted) to avoid state desynchronization.
- The algorithm must work in an asynchronous setting.

4.1. Architecture

The [AccessMgr](#) struct is the outer wrapper. It runs a single goroutine ([run](#)) that processes events from several channels¹:

- The set of trusted peers changes
- The access node list for a particular chain is updated
- A chain is deactivated on this node

¹[Pipes](#) are used instead of channels, to avoid blocking goroutines.

- [WaspChainAccessNodesV4.tla](#) – Models the access node management protocol (`GPA:AccessMgr`). Verifies that access nodes eventually learn which nodes will serve them (`ServerGetsKnown`).
- [WaspChainCmtLogSUI.tla](#) – Specifies the committee log adjusted to the SUI-based L1 (`CommitteeLog`). Ensures self-stabilization: crashed nodes can reboot and rejoin consensus without persistent internal state. Verifies `MaxLIWillDecide` and `NoRebootsAllDecide`.
- [WaspChainRecovery.tla](#) – Models committee recovery when more than F nodes have crashed (possibly losing persistent storage). Defines rules for rejoining without appearing byzantine.
- [WaspChainConsensus.tla](#) – Models the interaction of the consensus log with the L1 chain, including TX rejections and L1 reorganizations.
- [WaspChainConsensusJournal.tla](#) – Checks how lagging nodes can recover by their log indexes. Verifies that an $N-F$ quorum at the same `LogIndex` can complete ACS and advance.
- [WaspConsensusRecovery.tla](#) – Checks whether quorums are sufficient in the consensus protocol to ensure certificate-based recovery after a quorum assumption violation.
- [WaspDSSInsts.tla](#) – Models the use of DSS (Distributed Schnorr Signature) nonces across consensus rounds. Verifies that nonces are used exactly once and that faulty nodes cannot force others to drop nonces.
- [WaspACSS.tla](#) – Specifies Asynchronous Complete Secret Sharing (ACSS), abstracting away cryptographic details.
- [RBC.tla](#) – Models Reliable Broadcast as a black-box based on its standard properties (validity, integrity, termination, uniform agreement).
- [BrachaRBC.tla](#) – Specifies Bracha’s Reliable Broadcast algorithm with support for dynamic monotonic predicates, based on the original Bracha 1987 paper and the Das, Xiang & Ren variant.
- [IscBatchTimestamp.tla](#) – Models batch timestamp selection. Verifies that timestamps are non-decreasing along a chain when taking the maximum of proposed timestamps while excluding the F highest values.
- [WaspByzEnv.tla](#) – Shared helper module defining standard BFT node sets and quorum sizes ($F+1$, $2F+1$, $N-F$) under the assumption $N > 3F$. Extended by most other specifications.

Listing 2: TLA+ specifications in the ISC repository.

Each of these translates into a GPA Input that is fed into the inner `GPA:AccessMgr` instance. Network messages from peers are similarly received via a channel and passed as GPA `Message` calls. Outgoing messages produced by the GPA are serialized and sent over the peering network.

4.2. AccessMgr GPA

The inner GPA component is implemented in [access_manager_dist.go](#) and has a formal TLA+ specification in [WaspChainAccessNodesV4.tla](#).

4.2.1. State

The GPA maintains two maps:

- **Per-node state** (`accessMgrNode`): for each trusted peer, tracks:
 - `accessFor`: the set of chains for which we have granted access to that peer (i.e. we act as server).
 - `serverFor`: the set of chains for which *the peer* has granted access to us (i.e. they act as server).
 - `ourLC` / `peerLC`: logical clocks used to detect stale messages and drive convergence.
- **Per-chain state** (`accessMgrChain`): for each active chain, tracks:
 - `access`: which nodes have been granted access.
 - `server`: which nodes have confirmed they will serve us (i.e. the nodes that consider *us* an access node for this chain).

4.2.2. Inputs

The GPA accepts three kinds of inputs, corresponding to TLA+ spec actions:

1. **inputAccessNodes** (`input_access_nodes.go`): the access node list for a chain has been updated. The first reception of this input for a chain implicitly *activates* the chain (corresponding to `ChainActivate` in the spec). Subsequent receptions correspond to `AccessNodeAdd` / `AccessNodeDel`. For each trusted peer, a `msgAccess` message is sent with the updated access grants.
2. **inputChainDisabled** (`input_chain_disabled.go`): the chain has been deactivated. All peers are notified that access is revoked (`ChainDeactivate` in the spec).
3. **inputTrustedNodes** (`input_trusted_nodes.go`): the set of trusted peers has changed. New peers are initialized and sent the current access state; removed peers are disconnected and their server status is cleared. This corresponds to `Reboot` in the spec.

4.2.3. Message exchange

All peer-to-peer communication uses a single message type, `msgAccess`, which carries four fields:

- `senderLClock`: the sender's logical clock (version of its access grants).
- `receiverLClock`: the last known logical clock of the receiver (acts as an acknowledgment).
- `accessForChains`: the set of chains for which the sender grants access to the receiver.
- `serverForChains`: the set of chains for which the sender believes the receiver grants access to the sender (echo / ack).

4.2.4. Convergence

The protocol converges through a simple echo/ack mechanism driven by logical clocks (`handleMsgAccess`). When a node receives a message, it checks whether:

1. The peer's access information matches what we already know (`serverFor`).
2. The echoed server information matches our current access grants (`accessFor`).
3. The logical clocks are consistent (not outdated and not exceeding ours).

If all conditions hold, the state is converged for this peer and no reply is needed. Otherwise, a reply is sent with the node's current state, incrementing the logical clock if necessary. This ensures that after all in-flight messages are delivered, both sides agree on who serves whom. The liveness property proved in the TLA+ spec (`ServerGetsKnown`) states that if node *n* has granted access to node *a* for chain *c*, and both have the chain active, then eventually *a* will learn that *n* is a server for it.

When the set of server nodes for a chain changes, a callback (`serversUpdatedCB`) is invoked, which propagates the updated server list to the rest of the system (e.g. so the node knows where to send its queries). Note that the use of callbacks by the GPA component violates the pseudo-functional design principle of the GPA framework.

5. ChainMgr

[ChainMgr](#) is a per-chain GPA that acts as the central coordinator for a single chain within a node. Its main responsibilities are:

- **Track the correct branch:** maintain the latest confirmed anchor received from L1 and determine which state the chain should build upon.
- **Manage committee logs:** maintain one [CommitteeLog](#) instance for each committee this node participates in, and decide which committee is the *active* one.
- **Drive consensus:** based on [CommitteeLog](#) output, signal when a new consensus instance is needed ([NeedConsensus](#)) and which base anchor it should use.
- **Publish transactions:** collect signed transactions produced by consensus and request their publication to L1 ([NeedPublishTX](#)).
- **Inform access nodes:** when a new block is produced, send it to access nodes (via [msgBlockProduced](#)) so they can update their active state without waiting for L1 confirmation.

5.1. State

The key variables tracked by [ChainMgr](#) (documented in detail in the [source file header](#)) are:

- [latestConfirmedAnchor](#): the most recent anchor confirmed on L1. Updated whenever a new confirmed anchor is received from the node connection.
- [latestActiveCommittee](#): the committee address of the currently active committee. (The node could be participating in multiple committees e.g. in case of an ongoing rotation).
- [needConsensus](#): a map of consensus instances that need to be started, keyed by ([committeeAddr](#), [logIndex](#)). Populated from the output of some [CommitteeLog GPA instance](#).
- [needPublishTX](#): a map of transactions awaiting publication to L1. Entries are added when [Consensus](#) completes and removed when a publish result is received.
- [committeeLogs](#): a map of [committeeLogInst](#) structs, each wrapping a [CommitteeLog GPA instance](#) and its associated committee address and DK share.

5.2. Inputs

[ChainMgr](#) accepts the following inputs:

- [inputAnchorConfirmed](#): a new anchor has been confirmed on L1. [ChainMgr](#) updates [latestConfirmedAnchor](#), looks up the committee for the new state controller, and forwards the anchor to the corresponding [CommitteeLog](#). If this node is not in the new committee (in case of rotation), the previously active committee (if any) is suspended.
- [inputChainTxPublishResult](#): the result of publishing a transaction to L1. The transaction is removed from [needPublishTX](#). If the transaction was rejected, the result is forwarded to the [CommitteeLog](#) so it can recover.
- [inputConsensusOutputDone](#): a consensus instance completed successfully and produced a signed transaction. The transaction is added to [needPublishTX](#), and the produced block is sent to access nodes via [msgBlockProduced](#).
- [inputConsensusOutputSkip](#): a consensus instance decided to skip. Forwarded to the corresponding [CommitteeLog](#).
- [inputConsensusTimeout](#): a consensus instance timed out. Forwarded to the corresponding [CommitteeLog](#) so it can advance its log index.
- [inputCanPropose](#): signals that conditions are met to propose a new block (e.g. the Mempool has pending requests). Broadcast to all [CommitteeLog](#) instances.

5.3. Messages

ChainMgr exchanges two message types with peers:

- **msgCommitteeLog**: a wrapper that carries a CommitteeLog message and a committee address. When a CommitteeLog produces outgoing messages, ChainMgr wraps them with the committee address so the receiver can route them to the correct CommitteeLog instance.
- **msgBlockProduced**: sent by committee nodes to access nodes after consensus completes. It carries the signed transaction and the produced block, allowing access nodes to update their state before the transaction is confirmed on L1.

5.4. HandleCommitteeLogOutput

The **handleCommitteeLogOutput** method is invoked whenever a CommitteeLog instance produces output. This method decides whether the active committee should change and whether new Consensus instances are needed:

1. If the reporting committee is the active committee (or no committee is active yet), it becomes (or remains) the active committee and needConsensus is updated from its output.
2. If the reporting committee is *different* from the active one and its output requests a new consensus, the previously active committee is suspended and the reporting committee takes over.
3. If the reporting committee is different but has no consensus request, nothing changes – there is no reason to switch.

This logic ensures that committee rotations (triggered by L1 governance changes) are handled smoothly: the old committee is suspended and the new one takes over as soon as it has work to do.

5.5. CommitteeLog

CommitteeLog is the GPA responsible for producing a *log of block decisions* for a given chain. There is one CommitteeLog instance for each committee this node participates in (managed by ChainMgr). Its two main functions are:

1. **Propose when to start a consensus instance**, identified by a **Log Index** (LI), a monotonically increasing number that is kept in sync across the committee.
2. **Propose the base anchor for that consensus instance.**

The algorithm has a formal TLA+ specification in **WaspChainCmtLogSUI.tla**.

CommitteeLog is composed of three internal subcomponents that interact via callbacks:

1. **VarLogIndex**: Determines the current log index the committee should work on. It collects **MsgNextLogIndex** votes from peers using a **QuorumCounter**. When F+1 nodes report that they have started consensus at a given LI, the node advances its own agreedLI to match. On boot, the node starts from $\text{minLI} = \text{persistedLI} + 1$ so it never replays a log index it may have already participated in.
2. **VarLocalView**: Tracks the latest confirmed anchor from L1. When the tip anchor changes, it notifies VarConsInsts so that a concrete anchor can be proposed.
3. **VarConsInsts**: Maintains a sliding window of consensus instances (keyed by LI), each mapped to either an anchor or $\perp$. It receives signals from the other two subcomponents:
 - From VarLogIndex: the latest LI seen by a quorum (LatestSeenLI).
 - From VarLocalView: the latest confirmed L1 anchor (LatestL1Anchor).

When a consensus instance completes, VarConsInsts advances the window:

- **Done** (ConsOutputDone): the produced anchor becomes the input for LI+1.

- **Skip** (ConsOutputSkip): no block was produced; LI+1 gets the latest L1 anchor (or waits for one).
- **Timeout** (ConsOutputTimeout): LI+1 is proposed with $\perp$, signaling uncertainty.

The current window of (LI $\rightarrow$ anchor | $\perp$) entries is published as the [CommitteeLog output](#), which ChainMgr reads to populate its NeedConsensus map.

6. StateMgr

[StateMgr](#) is responsible for ensuring that the node has all the blocks it needs to participate in the chain. When a consensus instance starts, it may reference a base anchor whose corresponding block is not yet in the local store (e.g.

after a reboot, or when the node is lagging behind). StateMgr fetches the missing blocks from peer nodes, commits them to the store, and notifies the requesting component (Consensus, Mempool) when the state is ready.

There is one StateMgr per chain, managed by chainNodeImpl.

6.1. Outer wrapper

The [stateManager](#) struct is the outer wrapper. Like AccessMgr, it runs a single goroutine that processes events from several pipes:

- **Input pipe:** carries StateMgr GPA inputs coming from Consensus and Mempool.
- **Message pipe:** carries StateMgr GPA messages (block requests and responses).
- **Node public keys pipe:** updates the set of peers available for block fetching (server nodes, committee nodes).
- **Preliminary block pipe:** receives blocks forwarded by ChainMgr's msgBlockProduced mechanism, saving them to the WAL.

The wrapper also manages a [Block WAL](#) (write-ahead log) for crash recovery, a [block cache](#) for recently seen blocks, and a [snapshot manager](#) for loading snapshots on boot.

6.2. StateMgr GPA

Blocks are not stored on L1 – only anchors (commitments) are. When a node needs to reconstruct the chain state, it must obtain the actual block data from other nodes. The [StateMgr GPA subcomponent](#) encapsulates this block-fetching logic as a distributed protocol: it knows which blocks are missing, asks peers for them, and commits them to the local store in the correct order.

Its core mechanism is *block chain tracing*: given a target block commitment, it walks backwards through the chain of PreviousL1Commitment pointers until it finds a block that is already in the store. Any missing blocks along the way are queued for fetching from peers. Once all blocks in the chain are available, they are committed to the store in order.

6.2.1. Inputs

Note: some of the inputs accepted by StateMgr serve as a request-response mechanism for other components, which violates the pseudo-functional design principle of GPA.

- [ConsensusStateProposal](#): [Consensus](#) asks StateMgr to ensure all blocks up to a given anchor are available. Responds (via a channel) once the chain is complete.
- [ConsensusDecidedState](#): [Consensus](#) asks for the chain state at a given commitment. Responds with the state once all blocks are committed.
- [ConsensusBlockProduced](#): [Consensus](#) has produced a new block (state draft). StateMgr commits it to the store and adds it to the cache.

- **ChainFetchStateDiff**: the Mempool asks for the state difference between two anchors (used to determine which requests are already processed). Responds with the diff once both states are available.
- **StateManagerTimerTick**: periodic tick that retries pending GetBlock requests, cleans expired cache entries, and removes stale callbacks.

6.2.2. Messages

- **GetBlockMessage**: requests a block identified by its L1 commitment from a peer. Sent to a random subset of known peers.
- **BlockMessage**: a response carrying the requested block. Upon receipt, the block is added to the cache and the block chain tracing continues from where it left off.

7. Mempool

The **Mempool** manages the pool of pending requests for a single chain. Its main responsibilities are:

- **Provide proposal sets for consensus**: when a **Consensus** instance starts, it asks the Mempool for a set of request refs to include in the next block.
- **Provide resolved requests**: after Consensus decides on a set of request refs, it asks the Mempool for the actual request objects.
- **Share off-ledger requests between nodes**: off-ledger requests arrive via the API at a single node and must be disseminated to the committee and server nodes so they can participate in Consensus. This is the job of the Mempool GPA subcomponent.
- **Track the chain head**: when the chain advances (or rolls back), the Mempool adjusts its request set accordingly – removing processed requests, re-adding requests from reverted blocks, and refreshing on-ledger requests from L1 if needed.

There is one Mempool per chain, managed by `chainNodeImpl`.

7.1. Outer wrapper

The `mempoolImpl` struct is the outer wrapper. It runs a single goroutine (`run`) processing events from pipes for: tracking new chain heads, receiving on-ledger and off-ledger requests, consensus proposal/request queries, server/access node updates, and network messages. It maintains two request pools:

- **typedPool** (used for on-ledger requests): stores requests keyed by their ref, with type-specific limits and eviction policies.
- **offLedgerPool**: stores off-ledger requests with per-account limits and nonce-based ordering.

7.2. Mempool GPA

The inner GPA is implemented in `dist_sync.go`. Its purpose is to ensure that all committee nodes have the same off-ledger requests available for Consensus. Without this component, an off-ledger request submitted to a single node's API would never reach the other committee members, and Consensus would fail to include it.

7.2.1. Inputs

- **inputPublishRequest**: when a new off-ledger request is received locally, it is sent to all committee nodes (or all server nodes if the committee is not yet known) via `msgShareRequest`.
- **inputRequestNeeded**: when Consensus references a request that this node does not have, a `msgMissingRequest` is sent to all committee nodes. On subsequent timer ticks, if the request is still missing, queries are sent to random server nodes.
- **inputServerNodes**: the set of server and committee nodes has been updated.

- [inputAccessNodes](#): the set of access and committee nodes has been updated.

7.2.2. Messages

- [msgShareRequest](#): carries a full request object. The recipient adds the request to the local pool. If it was received from outside the committee and is new to this node, it is re-propagated to the committee to ensure full dissemination.
- [msgMissingRequest](#): carries a request reference. The recipient replies [msgShareRequest](#) if it has it.

8. Consensus

In ISC, the “Consensus” is the protocol that the nodes in a committee run to agree on the next block to be added to the chain.

The ISC Consensus is loosely based on the Honey Badger BFT protocol [2]. It is highly recommended to read the original paper, although we will describe the main ideas and components here.

Some further documentation about ISC Consensus can also be found in [this draft document](#) by Karolis Petrauskas, the main author of the Consensus code. Even if the document corresponds to a previous version of the ISC protocol, most of the ideas and components are still relevant.

In the Honey Badger paper, it is stated that it is an “asynchronous BFT” protocol. BFT stands for [Byzantine Fault Tolerant](#), which means that the protocol is resilient to a certain number f of faulty nodes in the committee of n nodes. Asynchronous means that the protocol guarantees liveness without making any timing assumptions (i.e. network delays, ordering of messages, etc).

Figure 2 shows a high-level view of the ISC Consensus protocol. The [consensusrunner](#) component is the main orchestrator that communicates with other components shown in blue (State Manager, Mempool, etc.), and controls the [Consensus GPA](#) instance.

A Consensus instance (with its corresponding consensusrunner) is spawned whenever the ChainMgr determines that a new block needs to be computed and added to the chain. In order to compute the new block we need:

- The **base L1 anchor**² (i.e. the latest chain state)
- The current **timestamp**
- The latest **L1 parameters** such as the base coin address, reference gas price, etc.
- A **list of requests** to execute (sometimes called a “batch”)
- A way to **sign** the produced block, so that it can be anchored on L1

1 When spawning the Consensus, the ChainMgr supplies the **proposed base anchor** as the first input to the Consensus GPA, although after the Consensus is executed, the **decided base anchor** [may be different](#) if not every node in the committee proposes the same base anchor.

2 The ChainMgr also [provides](#) the **timestamp** as an input to the Consensus GPA.

3 In order to obtain the L1 parameters, the consensusrunner [asks](#) the NodeConnection component to provide it. When this information is ready, it is provided as an input to the Consensus GPA.

4 In similar fashion, the consensusrunner:

²Instead of “anchor”, some old documentation may mention an “Alias Output” or “AO”, which corresponds to a previous version of ISC.

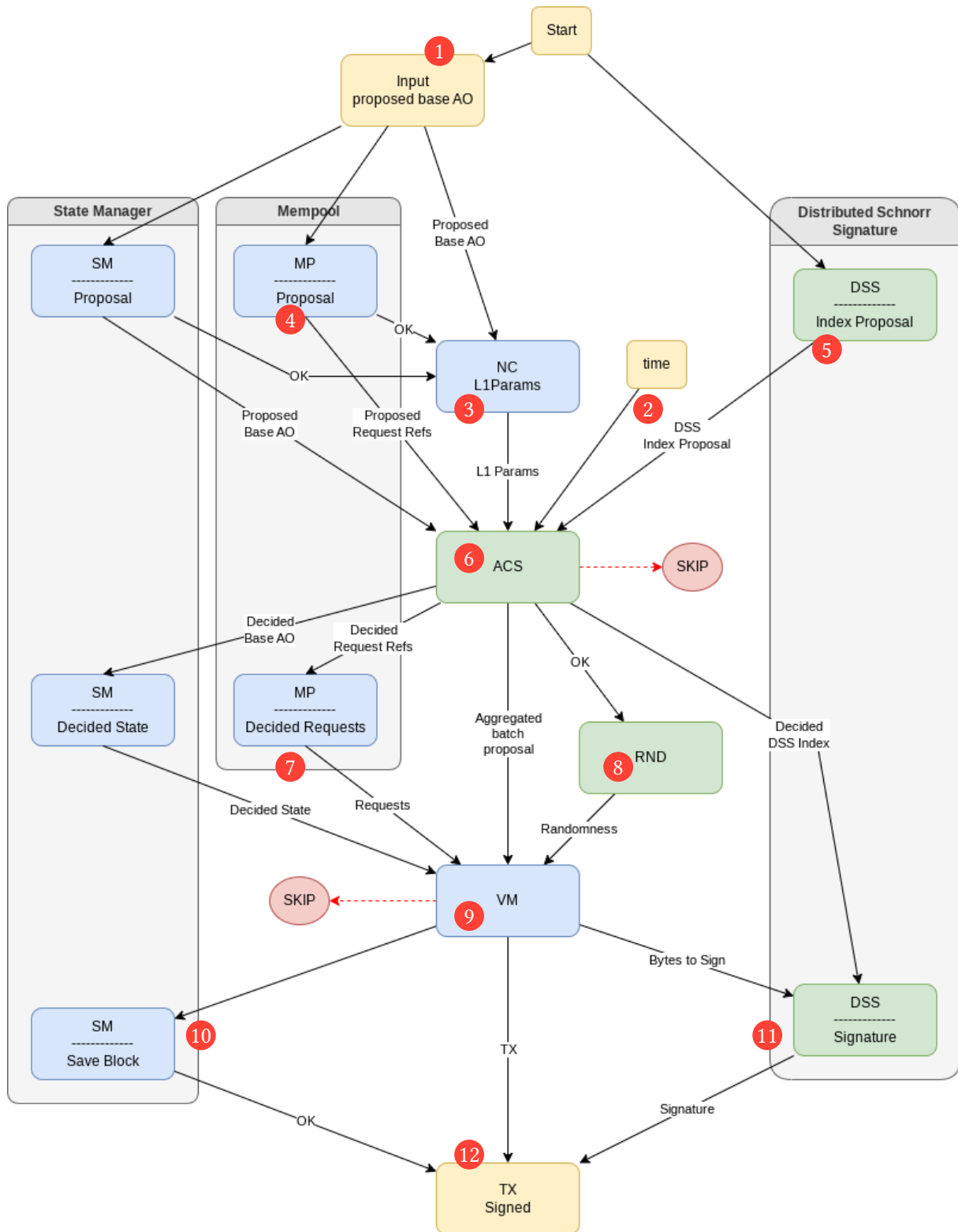


Figure 2: High-level view of the consensus protocol

- asks the StateMgr to provide the chain state corresponding to the proposed base anchor, and when it is ready, it is provided as an input to the Consensus GPA.

- asks the Mempool to provide a list of proposed request IDs, and when it is ready, it is provided as an input to the Consensus GPA.

5 While the Consensus inputs are being fetched, the Consensus GPA **starts** an instance of the **DSS protocol**, which is ultimately responsible for **signing** the produced block. The DSS protocol is designed so that $n - f$ honest nodes are enough to sign the block. Each node proposes a set of nodes (shown as “index proposal” in Figure 2) and after collecting the proposals from all peers, the DSS protocol will decide on an aggregated set of nodes that will sign the block.

6 With all needed inputs in place, an **ACS protocol is started**. ACS stands for “Asynchronous Common Subset”, and it is the main component of the ISC Consensus protocol. It is responsible for taking all the inputs from the different nodes in the committee, and deciding (agreeing) on a common subset of them, so that every honest node in the committee will end up with the same subset of inputs.

The output of ACS will be either a valid **aggregated batch** to compute a new block, or a **skip** action meaning that no block will be produced (e.g. if there are no valid requests). In case of a decided skip action, no further steps are needed by the Consensus.

7 Given the decided base anchor and request refs, the consensusrunner:

- asks the StateMgr to provide the chain state corresponding to the decided base anchor
- asks the Mempool to provide the requests corresponding to the decided request refs

8 In order to compute the new block, an **entropy** or **randomness** value is needed. The entropy value is a []byte that is used as a source of randomness for the block computation, and it is also included in the block itself. The entropy value is **computed** as the BLS signature of the base anchor’s reference. This way, the entropy value is unique to each anchor version, and unpredictable by any party that is not part of the committee.

9 From all the gathered inputs, the Consensus **creates** a VMTask that is **used** to run a VM instance. The output of the VM is either a valid **block**, or a **skip** action. In case of a decided skip action, no further steps are needed by the Consensus.

10 As a result of the VM run there is a new chain state that needs to be persisted in the DB. The StateMgr is **asked** to perform this action, and when it is done, the new state is ready to be anchored on L1.

11 The produced block is wrapped into a **L1 transaction** that needs to be **signed** by the committee. The TX bytes is **passed** as an input to DSS, along with the sets of nodes proposed by each peer.

12 **Finally**, the TX is signed and the Consensus is finished.

8.1. DSS

The **DSS (Distributed Schnorr Signature) protocol** is responsible for signing the L1 transaction that anchors the new block. It is designed so that the honest nodes in the committee can sign the block even in the presence of f faulty nodes. The signing algorithm is described in [3].

To generate a distributed signature, the protocol requires one longterm distributed secret (generated as part of the chain creation process), and one random secret to be used only once (generated for each signing operation). Each participant then issues a partial signature that can be broadcast to the

whole group. Once one has collected enough partial signatures, it is possible to compute the distributed signature.

1. [Start a NonceDKG](#) GPA instance, in order to generate the distributed random secret. See Section 8.2 for more information about this protocol. NonceDKG will produce an intermediate output (a list of proposed honest nodes), and a final output (the random secret to be used for DSS).
2. When the NonceDKG produces the intermediate output (the list of proposed nodes), forward it as an [intermediate output of DSS](#).
3. [Wait for input](#) containing:
 - sets of nodes proposed by each peer
 - TX bytes to be signed,

Then forward the sets of nodes as an input to NonceDKG.

4. When NonceDKG [produces](#) its final output (the random secret), create a partial signature of the TX bytes, and broadcast it to all peers.
5. When [enough partial signatures are collected](#), compute the final distributed signature, and output it as the result of DSS.

8.2. NonceDKG

[NonceDKG](#) is a distributed key generation protocol that is used to generate the random secret needed for the DSS protocol. NonceDKG is adapted from [4].

1. Each node [generates](#) a random secret value and broadcasts it to all peers using the [ACSS protocol](#) (which guarantees that only honest nodes can recover the shared secrets). Thus, each NonceDKG instance manages n instances of ACSS, one for broadcasting, and $n - 1$ for receiving the secrets from peers.
2. As soon as the secrets are [received from \$n - f\$ peers](#), these peers are proposed as the set of nodes that will sign the block. This is an intermediate output of NonceDKG.
3. Wait for input containing the sets of nodes proposed by each peer. Then [produce the final output](#), containing:
 - an aggregated set of nodes
 - the random secret to be used for DSS (computed as the sum of the secrets proposed by the nodes in the aggregated set)

8.3. ACSS

[ACSS](#) is an implementation of the Asynchronous Complete Secret Sharing protocol, which is used as a building block for the NonceDKG protocol. ACSS is adapted from [5], [6].

“Secret Sharing” means that a secret value is split into n shares, and distributed among n nodes in such a way that any subset of $f+1$ shares can be used to reconstruct the secret, but any subset of f shares cannot learn anything about the secret.

“Complete” means that the protocol guarantees one of two outcomes:

1. All honest nodes can reconstruct the same secret.
2. They collectively agree that the secret cannot be reconstructed (e.g. because the dealer is faulty and did not share the secret correctly).

The algorithm pseudocode is [shown](#) at the top of the source code. The main idea is that the “dealer” (the node that wants to share the secret) splits the secret into n shares, and encrypts each share using the public key of the corresponding peer. Then it broadcasts all the shares using the [RBC](#)

[protocol](#). After that, the nodes attempt to decrypt the shares they receive, and inform each other about the success or failure. After some rounds of communication, the nodes can decide whether they have enough valid shares to reconstruct the secret, or if they collectively agree that the secret cannot be reconstructed.

Note: for a receiver node, the output of ACSS is the received share. The actual secret is never reconstructed inside ACSS.

8.4. RBC

[RBC](#) is an implementation of Bracha's Reliable Broadcast protocol. Its main purpose is to allow a node to broadcast a message to all peers in such a way that all honest nodes will receive the same message. RBC first appeared in [7], although our implementation is mainly taken from [8].

The pseudocode is [shown](#) at the top of the source code. The main idea is that the sender node broadcasts the message using a simple "unreliable" broadcast, and then the nodes exchange "echo" and "ready" messages to ensure that all honest nodes receive the same message.

8.5. ACS

[ACS](#) is an implementation of the Asynchronous Common Subset protocol, which is the main component of the ISC Consensus protocol. It is responsible for taking all the inputs from the different nodes in the committee, and deciding (agreeing) on a common subset of them, so that every honest node in the committee will end up with the same subset of inputs. ACS is mainly adapted from [2].

The pseudocode is [shown](#) at the top of the source code.

1. Each node i receives an input v_i , and broadcasts it to all peers using the [RBC protocol](#). Thus, there are n instances of RBC running in parallel, one for each node in the committee.
2. Each node also starts n instances of the [ABA protocol](#). Whenever RBC_j outputs v_j , the node provides input 1 to ABA_j . This essentially means that the node i "votes" for including v_j in the common subset.
3. When $n - f$ instances of ABA have input 1, provide input 0 to all ABAs without input. This ensures that all ABA instances will eventually produce an output.
4. Once all ABA instances have produced an output, the ones that have output 1 correspond to the inputs that are included in the common subset. The node can then output the common subset of inputs.

8.6. ABA

[ABA](#) is an implementation of the Asynchronous Binary Agreement protocol. Its purpose is to allow a group of nodes to agree on a common binary value (0 or 1). ABA first appeared in [9], although our implementation is mainly taken from [2].

The pseudocode is [shown](#) at the top of the source code. The input to ABA is a binary value (0 or 1) that the node wants to propose. The output is a binary value that is agreed upon by all honest nodes in the committee. The protocol proceeds in rounds, and in each round the nodes exchange their proposed values and a [common coin](#) value (which is a random value that is the same for all nodes in the round).

The protocol ensures that if all honest nodes propose the same value, then that value will be the output. If there is disagreement, the common coin is used to break ties and ensure progress.

8.7. CC

CC is an implementation of the Common Coin protocol based on BLS threshold signatures. A “Common Coin” is a random binary value (0 or 1) that is the same for all peers. This protocol is described in Appendix C of [2].

1. Each node takes as an input a “session ID” (a []byte), so that many CC instances can run in parallel without interfering with each other. When used as part of ABA, the session ID includes the Consensus LogIndex and the ABA round number.
2. Each node computes a partial BLS signature of the session ID using its share of the distributed key, and broadcasts it to all peers.
3. When enough partial signatures are collected, the final BLS signature is reconstructed, and the output 0 or 1 is the least significant bit of the signature.

References

- [1] M. Raynal, *Fault-tolerant message-passing distributed systems*, 1st ed. Cham, Switzerland: Springer International Publishing, 2018.
- [2] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, “The Honey Badger of BFT Protocols,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, in CCS '16. Vienna, Austria: Association for Computing Machinery, 2016, pp. 31–42. doi: [10.1145/2976749.2978399](https://doi.org/10.1145/2976749.2978399).
- [3] D. R. Stinson and R. Stroh, “Provably Secure Distributed Schnorr Signatures and a (t, n) Threshold Scheme for Implicit Certificates,” in *Proceedings of the 6th Australasian Conference on Information Security and Privacy*, in ACISP '01. Berlin, Heidelberg: Springer-Verlag, 2001, pp. 417–434.
- [4] S. Das, T. Yurek, Z. Xiang, A. Miller, L. Kokoris-Kogias, and L. Ren, “Practical Asynchronous Distributed Key Generation.” [Online]. Available: <https://eprint.iacr.org/2021/1591>
- [5] A. Kate, A. Miller, and T. Yurek, “Brief Note: Asynchronous Verifiable Secret Sharing with Optimal Resilience and Linear Amortized Overhead.” [Online]. Available: <https://arxiv.org/abs/1902.06095>
- [6] T. Yurek, L. Luo, J. Fairoze, A. Kate, and A. Miller, “hbACSS: How to Robustly Share Many Secrets.” [Online]. Available: <https://eprint.iacr.org/2021/159>
- [7] G. Bracha, “Asynchronous Byzantine agreement protocols,” *Information and Computation*, vol. 75, no. 2, pp. 130–143, 1987, doi: [https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/10.1016/0890-5401(87)90054-X).
- [8] S. Das, Z. Xiang, and L. Ren, “Asynchronous Data Dissemination and its Applications,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, in CCS '21. Virtual Event, Republic of Korea: Association for Computing Machinery, 2021, pp. 2705–2721. doi: [10.1145/3460120.3484808](https://doi.org/10.1145/3460120.3484808).
- [9] A. Mostéfaoui, H. Moumen, and M. Raynal, “Signature-Free Asynchronous Binary Byzantine Consensus with $t < n/3$, $O(n^2)$ Messages, and $O(1)$ Expected Time,” *J. ACM*, vol. 62, no. 4, Sept. 2015, doi: [10.1145/2785953](https://doi.org/10.1145/2785953).